MAQ Software

C# Concepts

October 10, 2026

Microsoft
Preferred Vendor

Microsoft Partner
Gold Data Analytics
Gold Data Platform
Gold Application Development

bsi. ISO/IEC 27001 Information Security Management

Inc.5000
The Fastest Growing Private Companies in America

Agenda:

1. Introduction to C#
2. Program Structure
3. Fundamental Concepts
4. Data Types
5. Namespace
6. Loops
7. Interface
8. Decision Making
9. Getter and Setter Methods

Introduction to C#

- C# is a simple, modern, general-purpose, object-oriented programming language developed by Microsoft within its .NET initiative led by Anders Hejlsberg.
- Not all the supported languages fit entirely neatly into the .NET framework, but the one language that is guaranteed to fit in perfectly is C#.
- C# (C Sharp), a successor to C++, has been released in conjunction with the .NET framework.
- **C# design goals:**
  - Be comfortable for C++ programmer
  - Fit cleanly into the .NET Common Language Runtime (CLR)
  - Simplify the C++ model
  - Provide the right amount of flexibility
  - Support component-centric development

Program Structure

- **A C# program consists of the following parts –**
  - Namespace declaration
  - A Class
  - Class methods
  - Class attributes
  - A Main method
  - Statements and Expressions
  - Comments

Let us look at a simple code that prints the words "Hello World" –

- **It is worth to note the following points –**
  - C# is case sensitive.
  - All statements and expression must end with a semicolon (;).
  - The program execution starts at the Main method.
  - Unlike Java, program file name could be different from the class name.

```
using System;

namespace HelloWorldApplication {
   class HelloWorld {
      static void Main(string[] args) {
         /* my first program in C# */
         Console.WriteLine("Hello World");
         Console.ReadKey();
      }
   }
}
```

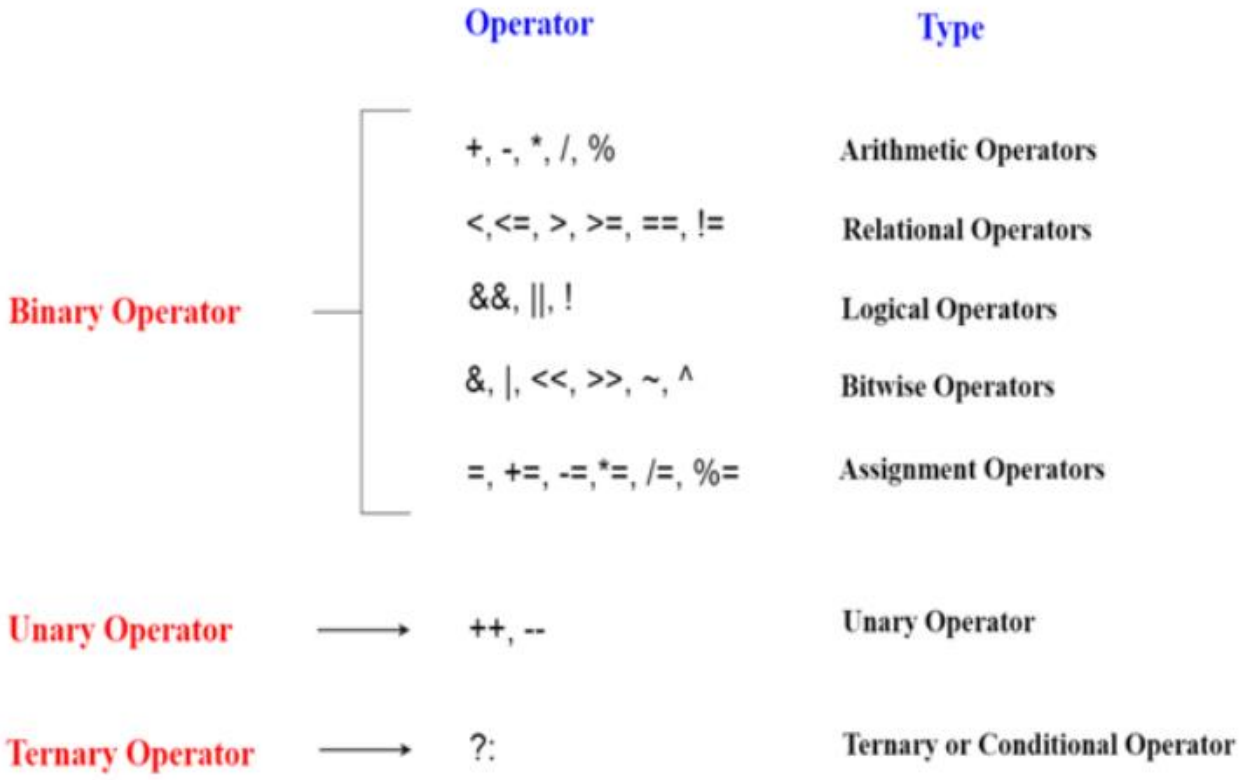
Fundamental Concepts

- **Variable :** Placeholder of the information which can be changed at runtime. Contains name, type and value. Initialized in two ways – compile time and run time.
- **Literals :** Literal is a value that is used by the variables.

Literal type	Examples
Integer Literals	101, 0146, 0x123Face, 0b101,
Floating-point Literals	3.14145, 312569E-5, 784f, 784d
Character Literals	‘a’. '\u0061’, ‘\n’
String Literals	“Hello World”, @”Hello World”
Null Literals	null
Boolean Literals	True, false

Fundamental Concepts

- **Operators :** An operator is a symbol that tells the compiler to perform specific mathematical or logical manipulations.

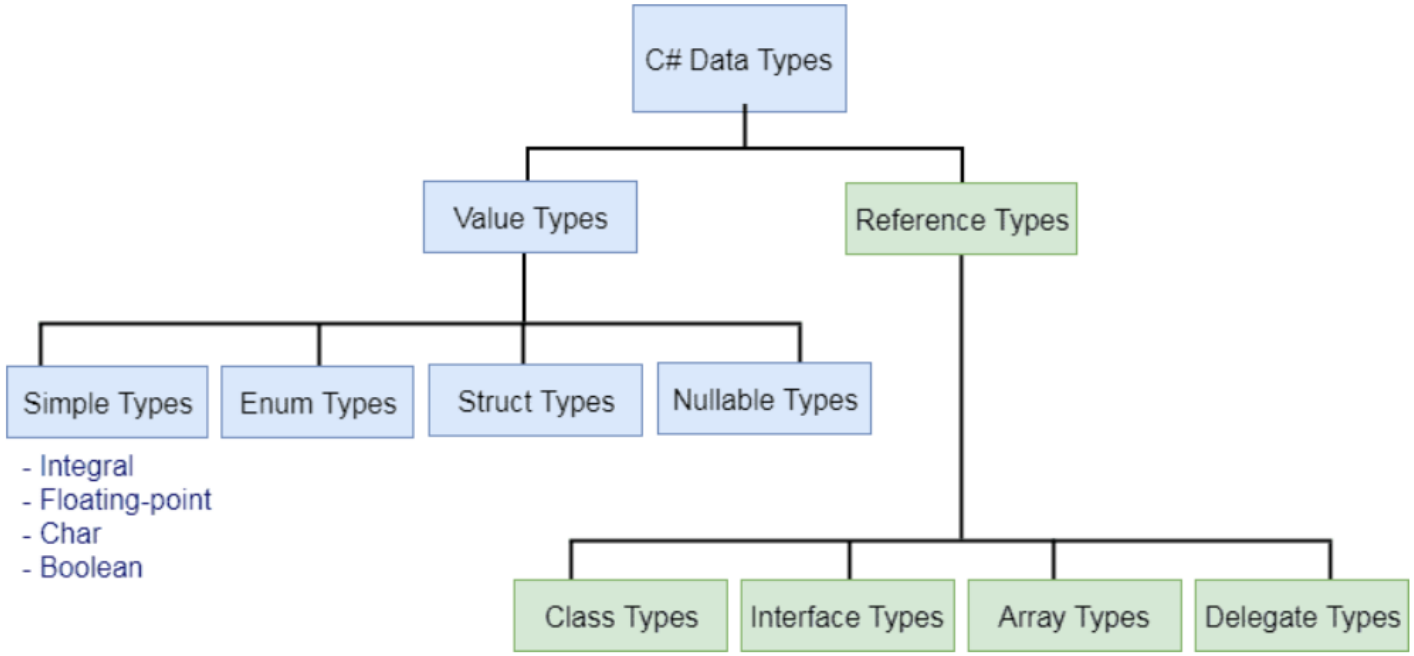


Fundamental Concepts

- **Keywords**: A keyword is a reserved word. You cannot use it as a variable name, constant name etc. if we want to use the keywords as identifiers, we may prefix the keyword with @ character.
- **Constants**: The constants refer to fixed values that the program may not alter during its execution. These fixed values are also called literals
- **Comments:** Comments can be used to explain C# code, and to make it more readable. There are two types of comments in C#:
    - Single line comment is defined with two forward slashes //
    - Multiple line comment starts with /*……. and ends with */

Data Types

- **Value type** variables can be assigned a value directly. They are derived from the class **System.ValueType**.
- The **reference types** do not contain the actual data stored in a variable, but they contain a reference to the variables.
- The **Object Type** is the ultimate base class for all data types in C# Common Type System (CTS). Object is an alias for System.Object class.



Namespaces

- A **namespace** is designed for providing a way to keep one set of names separate from another.
- The class names declared in one namespace does not conflict with the same class names declared in another.
- Defining a Namespace

```
namespace namespace_name {
       // code declarations
}
```

- To call the namespace-enabled version of either function or variable

```
namespace_name.item_name;
```

Namespaces

- **The *using* Keyword**

  The **using** keyword states that the program is using the names in the given namespace. For example, we are using the **System** namespace in our programs. The class Console is defined there. We just write −

```
Console.WriteLine ("Hello there");
```

  We could have written the fully qualified name as −

```
System.Console.WriteLine("Hello there");
```

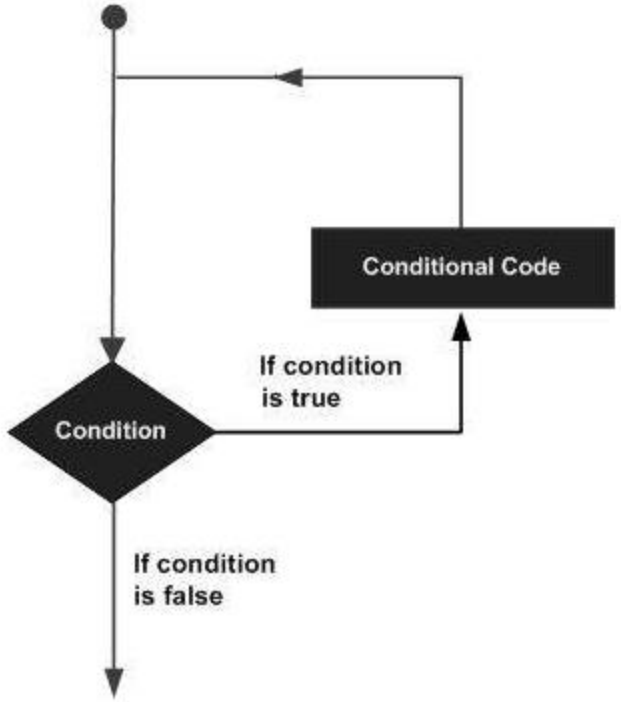
- ***Nested Namespaces***

```
namespace namespace_name1 {

   // code declarations
   namespace namespace_name2 {
      // code declarations
   }
}
```

Loops

A loop statement allows us to execute a statement or a group of statements multiple times and following is the general from of a loop statement in most of the programming languages −



Loops

C# provides following types of loop to handle looping requirements.

Sr.No.	Loop Type & Description
1	**while Loop:** It repeats a statement or a group of statements while a given condition is true. It tests the condition before executing the loop body.
2	**for loop:** It executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.
3	**do…while loop:** It is similar to a while statement, except that it tests the condition at the end of the loop body
4	**Nested loops:** You can use one or more loop inside any another while, for or do..while loop.
5	**Foreach** loop: Used to iterate over **collections** (arrays, lists, etc.)

Loops

For Loop:

```
for (int i = 1; i <= 5; i++)
{
  Console.WriteLine("Iteration: " + i);
}
```

Output:

```
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
```

While Loop:

```
int i = 1;
while (i <= 5)
{
  Console.WriteLine("Iteration: " + i); i++;
}
```

Output:

```
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
```

Loops

Do...While Loop:

```
int i = 1;
do
{
   Console.WriteLine("Iteration: " + i);
    i++;
}
 while (i <= 5);
```

Output:

```
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
```

Foreach Loop:

```
string[] names = { "Alice", "Bob", "Charlie" };
foreach (string name in names)
 {
    Console.WriteLine("Hello " + name);
}
```

Output:

```
Hello Alice
Hello Bob
Hello Charlie
```

Loops

Loop Control Statements

- Loop control statements change execution from its normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed.
- C# provides the following control statements.

Sr.No.	Control Statement & Description
1	**break statement:** Terminates the **loop** or **switch** statement and transfers execution to the statement immediately following the loop or switch.
2	**continue statement:** Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.

Infinite Loop

- A loop becomes infinite loop if a condition never becomes false. The for loop is traditionally used for this purpose.

Loops

Break statement:

```
for (int i = 1; i <= 5; i++)
{
  if (i == 3)
  {
    break; // exit loop when i == 3
  }
  Console.WriteLine("i = " + i);
}
Console.WriteLine("Loop ended");
```

Output:

```
i = 1
i = 2
Loop ended
```

Loops

Continue statement:

```
for (int i = 1; i <= 5; i++)
 {
     if (i == 3)
     {
       continue; // skip when i == 3
     }
 Console.WriteLine("i = " + i);
}
```

Output:

i = 1
i = 2
i = 4
i = 5

Interfaces

•An **interface** in C# is like a contract.
It defines what a class should do, but not how it does it.

•It can only contain method signatures, properties, events, or indexers (no implementation).
•A class or struct that implements an interface must provide the implementation of all its members.
•Supports multiple inheritance (a class can implement multiple interfaces).

```
public interface ITransactions {
    // interface members
    void showTransaction();
    double getAmount();
}
```

Interfaces

```
interface IAnimal
{
    void Speak(); // Only declaration
}
class Dog : IAnimal
{
    public void Speak() // Must implement
    {
        Console.WriteLine("Dog barks");
    }
}
class Cat : IAnimal
{
    public void Speak()
    {
        Console.WriteLine("Cat meows");
    }
}
```

```
class Program
{
    public static void Main()
    {
        IAnimal dog = new Dog();
        IAnimal cat = new Cat();
        dog.Speak(); // Dog barks
        cat.Speak(); // Cat meows
    }
}
```

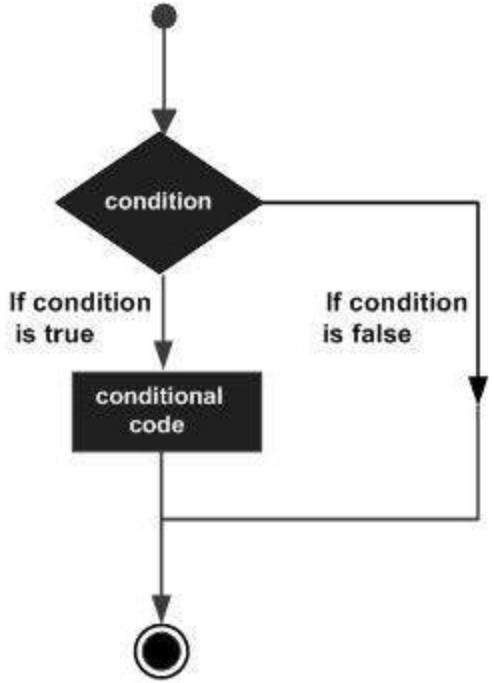
Output:

Dog barks

Cat meows

Decision Making

- Decision making structures requires the programmer to specify one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.
- Following is the general form of a typical decision-making structure found in most of the programming languages −



Decision Making

- C# provides following types of decision-making statements.

Sr.No.	Statement & Description
1	**If Statement:** An **if statement** consists of a Boolean expression followed by one or more statements.
2	**If… else Statement:** An **if statement** can be followed by an optional **else statement**, which executes when the Boolean expression is false.
3	**Nested if Statement:** You can use one **if** or **else if** statement inside another **if** or **else if** statement(s).
4	**Switch Statement:** A **switch** statement allows a variable to be tested for equality against a list of values.
5	**Nested Switch Statement:** You can use one **switch** statement inside another **switch** statement(s).

Getter and Setter methods

- **Getter method**: Returns the value of the variable
- **Setter method**: Assigns values to the variable

Advantages:

- Provide encapsulation to data
- Provides security to class properties
- Data validation before setting the new values

```
class Person
{
  private string name; // field
  public string Name   // property
  {
    get { return name; }
    set { name = value; }
  }
}
```

```
class Person
{
  private string name; // field
  public string Name   // property
  {
    get { return name; }
    set { name = value; }
  }
}
```

MAQ Software



Discussion

Microsoft Preferred Vendor
Microsoft Partner
Gold Data Analytics
Gold Data Platform
Gold Application Development
amazon web services Partner Network
CONSULTING PARTNER
bsi. ISO/IEC 27001 Information Security Management
Inc. 5000
The Fastest Growing Private Companies in America